What is gradient descent?

Gradient descent is an optimization algorithm which is commonly-used to train machine learning models and neural networks. It trains machine learning models by minimizing errors between predicted and actual results.

Training data helps these models learn over time, and the cost function within gradient descent specifically acts as a barometer, gauging its accuracy with each iteration of parameter updates. Until the function is close to or equal to zero, the model will continue to adjust its parameters to yield the smallest possible error.

Once machine learning models are optimized for accuracy, they can be powerful tools for artificial intelligence (AI) and computer science applications.

How does gradient descent work?

Before we dive into gradient descent, it may help to review some concepts from linear regression. You may recall the following formula for the slope of a line, which is y = mx + b, where *m* represents the slope and *b* is the intercept on the y-axis.

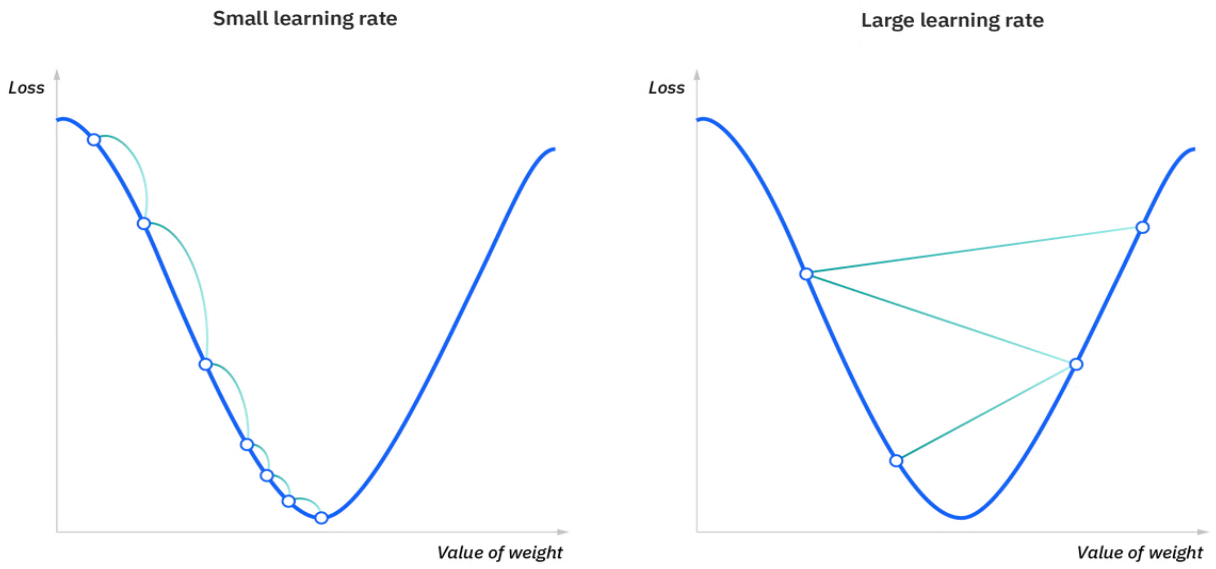
You may also recall plotting a scatterplot in statistics and finding the line of best fit, which required calculating the error between the actual output and the predicted output (y-hat) using the mean squared error formula. The gradient descent algorithm behaves similarly, but it is based on a convex function.

The starting point is just an arbitrary point for us to evaluate the performance. From that starting point, we will find the derivative (or slope), and from there, we can use a tangent line to observe the steepness of the slope. The slope will inform the updates to the model parameters—i.e. the weights and bias. The slope at the starting point will be steeper, but as new parameters are generated, the steepness should gradually reduce until it reaches the lowest point on the curve, known as the point of convergence.

Similar to finding the line of best fit in linear regression, the goal of gradient descent is to minimize the cost function, or the error between predicted and actual y. In order to do this, it requires two data points—a direction and a learning rate. These factors determine the partial derivative calculations of future iterations, allowing it to gradually arrive at the local or global minimum (i.e. point of convergence).

- Learning rate (also referred to as step size or the alpha) is the size of the steps that are taken to reach the minimum. This is typically a small value, and it is evaluated and updated based on the behavior of the cost function. High learning rates result in larger steps but risks overshooting the minimum. Conversely, a low learning rate has small step sizes. While it has the advantage of more precision, the number of iterations compromises overall efficiency as this takes more time and computations to reach the minimum.
- **The cost (or loss) function** measures the difference, or error, between actual y and predicted y at its current position. This improves the machine learning model's efficacy by providing feedback to the

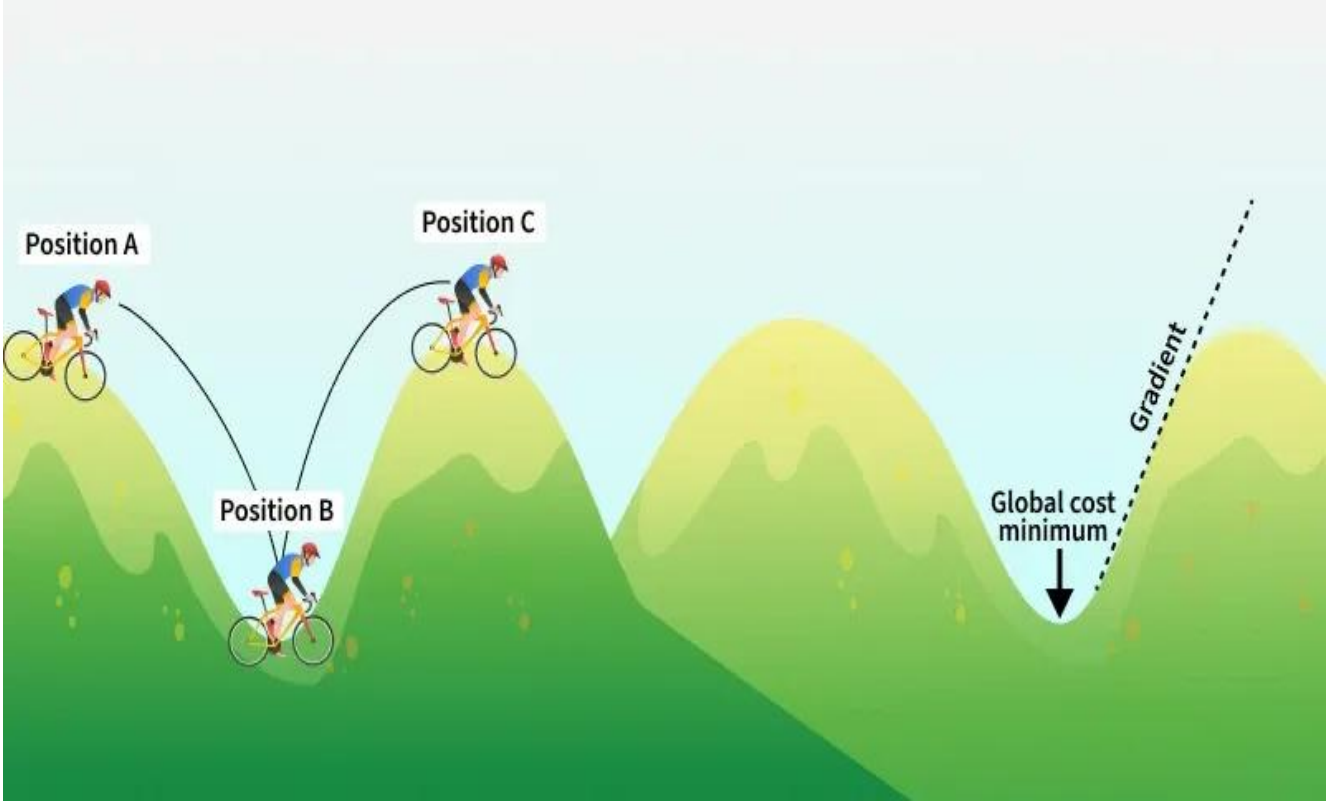
model so that it can adjust the parameters to minimize the error and find the local or global minimum. It continuously iterates, moving along the direction of steepest descent (or the negative gradient) until the cost function is close to or at zero. At this point, the model will stop learning. Additionally, while the terms, cost function and loss function, are considered synonymous, there is a slight difference between them. It's worth noting that a loss function refers to the error of one training example, while a cost function calculates the average error across an entire training set.



What is Gradient Descent

Gradient Descent is an iterative optimization algorithm used to minimize a cost function by adjusting model parameters in the direction of the steepest descent of the function's gradient. In simple terms, it finds the optimal values of weights and biases by gradually reducing the error between predicted and actual outputs.

For example:



Suppose you're at the top of a hill and your goal is to find the lowest point in the valley. You can't see the entire valley from the top, but you can feel the slope under your feet.

1. **Start at the Top:** You begin at the top of the hill (this is like starting with random guesses for the model's parameters).
2. **Feel the Slope:** You look around to find out which direction the ground is sloping down. This is like calculating the gradient, which tells you the steepest way downhill.
3. **Take a Step Down:** Move in the direction where the slope is steepest (this is adjusting the model's parameters). The bigger the slope, the bigger the step you take.
4. **Repeat:** You keep repeating the process feeling the slope and moving downhill until you reach the bottom of the valley (this is when the model has learned and minimized the error).

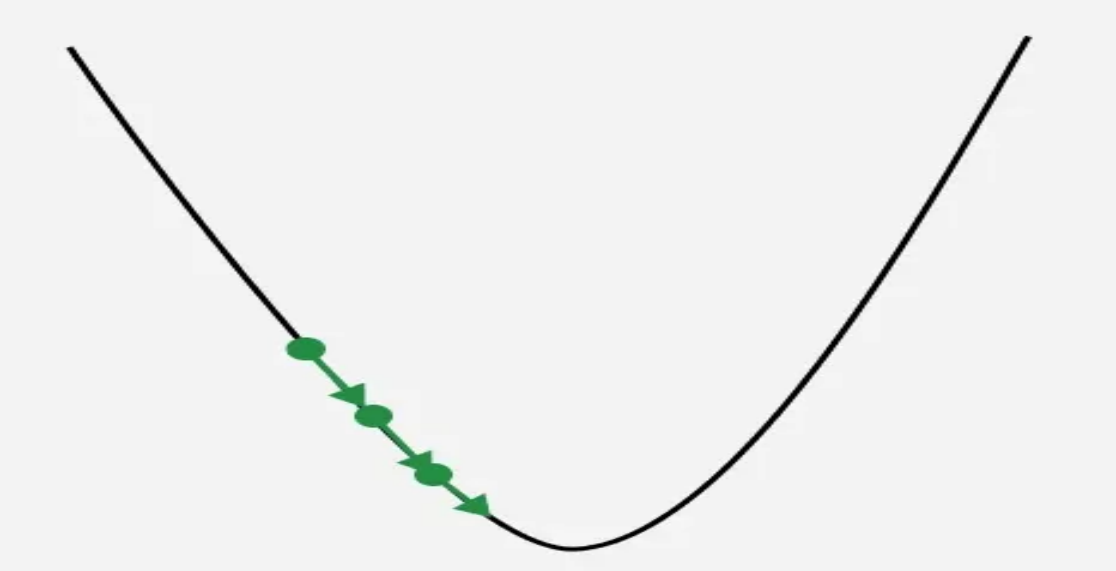
The key idea is that, just like walking down a hill, Gradient Descent moves towards the "bottom" or minimum of the loss function, which represents the error in predictions.

Moving in opposite direction of the gradient allows the algorithm to gradually descend towards lower values of the function and eventually reaching to the minimum of the function. These gradients guide the updates ensuring convergence towards the optimal parameter values. Gradual steps used in descent is done by defining learning rate.

What is Learning Rate?

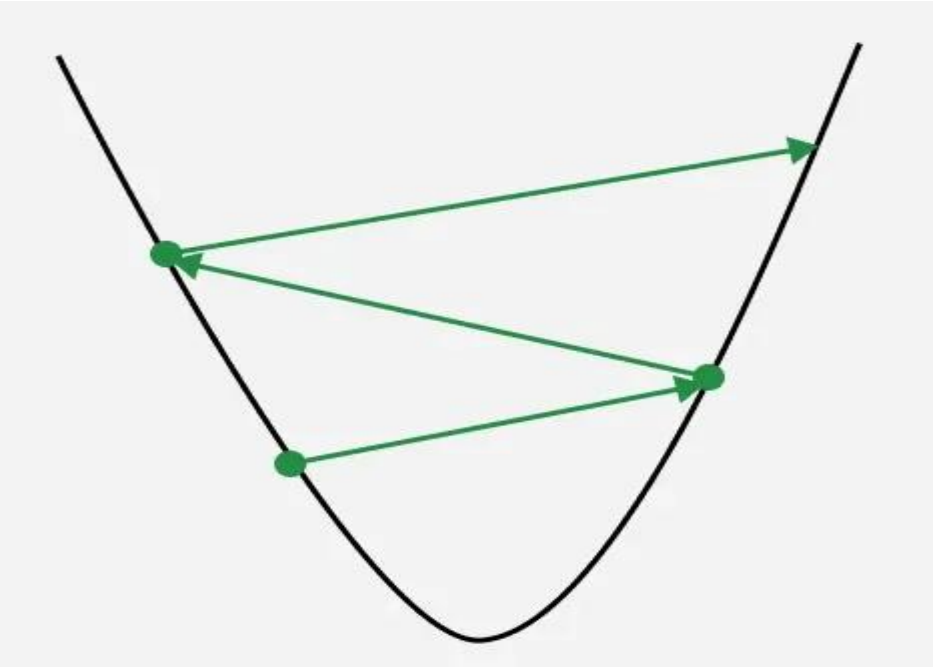
Learning rate is a important hyperparameter in gradient descent that controls how big or small the steps should be when going downwards in gradient for updating models parameters. It is essential to determines how quickly or slowly the algorithm converges toward minimum of cost function.

1. If Learning rate is too small: The algorithm will take tiny steps during iteration and converge very slowly. This can significantly increases training time and computational cost especially for large datasets.



Learning rate with small steps

2. If Learning rate is too big: The algorithm may take huge steps leading overshooting the minimum of cost function without settling. It fail to converge causing the algorithm to oscillate. This process is termed as exploding gradient problem.



In image we can see point got oscillated from right to left with converging to minimum gradient value.

To address these problems we have some technique that can be used:

- **Weights Regularzations:** The initialization of weights can be adjusted to ensure that they are in an appropriate range. Using a different activation function such as the Rectified Linear Unit (ReLU) can help us to mitigate the vanishing gradient problem.
- **Gradient clipping:** Restrict the gradients to a predefined range to prevent them from becoming excessively large or small.
- **Batch normalization:** It can also help to address these problems by normalizing the input of each layer to prevent activation function from saturating and hence reducing vanishing and exploding gradient problems.